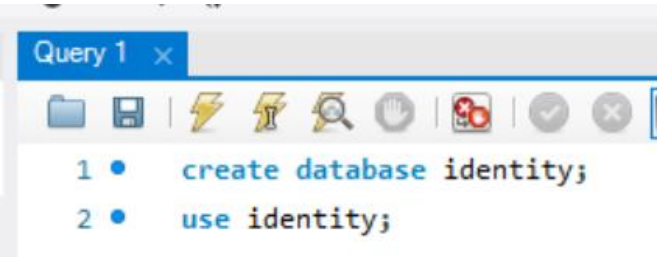
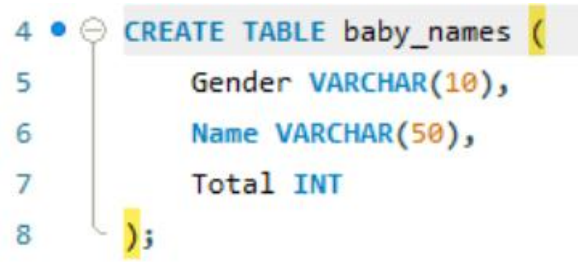
SQL: WINDOW FUNCTION & CTE (Common Table Expression)

1. Membuat database



2. Membuat table



3. Isi tabelnya

```
INSERT INTO baby_names (Gender, Name, Total) VALUES
('Girl', 'Ava', 95),
('Girl', 'Emma', 106),
('Boy', 'Ethan', 115),
('Girl', 'Isabella', 100),
('Boy', 'Jacob', 101),
('Boy', 'Liam', 84),
('Boy', 'Logan', 73),
('Boy', 'Noah', 120),
('Girl', 'Olivia', 100),
('Girl', 'Sophia', 88);
```

4. Lihat isi tabel

select * from baby_names;

Result Grid | Filter Rows:

Gender	Name	Total
Girl	Ava	95
Girl	Emma	106
Boy	Ethan	115
Girl	Isabella	100
Boy	Jacob	101
Boy	Liam	84
Boy	Logan	73
Boy	Noah	120
Girl	Olivia	100
Girl	Sophia	88

5. Diurutkan berdasarkan populasi
select *

from baby_names

order by total desc;

```
25      -- 2. ORDER by POPULARITY/TOTAL
26  •   select *
27      from baby_names
28      order by total desc;
29
```

Result Grid | Filter Rows:

Gender	Name	Total
Boy	Noah	120
Boy	Ethan	115
Girl	Emma	106
Boy	Jacob	101
Girl	Isabella	100
Girl	Olivia	100
Girl	Ava	95
Girl	Sophia	88
Boy	Liam	84
Boy	Logan	73

6. Menambahkan kolom populasi

Di sini kita dapat menambahkan Window Function.

Window dengan klausa OVER()

Dan Function adalah ROW NUMBER()

ROW NUMBER tidak bisa berdiri sendiri, ia harus bersama klausa OVER()

Jadi, window function ini berguna untuk mengurutkan ranking dengan menambahkan kolom baru di sebelahnya.

select Gender, Name, Total,

row_number() over(order by total desc) as Popularity

from baby_names;

```
30      -- 3. ADD POPULARITY COLUMN
31      select Gender, Name, Total,
32             row_number() over(order by total desc) as Popularity
33      from baby_names;
34
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Gender	Name	Total	Popularity
Boy	Noah	120	1
Boy	Ethan	115	2
Girl	Emma	106	3
Boy	Jacob	101	4
Girl	Isabella	100	5
Girl	Olivia	100	6
Girl	Ava	95	7
Girl	Sophia	88	8
Boy	Liam	84	9
Boy	Logan	73	10

Dengan cara CTE:

```
36  WITH popularity AS (
37      SELECT Gender,
38             Name,
39             Total,
40             ROW_NUMBER() OVER(
41                 PARTITION BY Gender
42                 ORDER BY Total DESC
43             ) AS Popularity
44      FROM baby_names
45  )
46  SELECT *
47  FROM popularity;
```

7. Mencoba function yang berbeda

Kita dapat menggunakan jenis function yang lain seperti RANK dan DENSE_RANK.

Contoh:

select Gender, Name, Total,

row_number() over(order by total desc) as Popularity,

rank() over(order by total desc) as Popularity_R,

dense_rank() over(order by total desc) as Popularity_DR

from baby_names;

```
35      -- 4. TRY DIFFERENT FUNCTIONS
36  •   select Gender, Name, Total,
37          row_number() over(order by total desc) as Popularity,
38          rank() over(order by total desc) as Popularity_R,
39          dense_rank() over(order by total desc) as Popularity_DR
40      from baby_names;
41
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Gender	Name	Total	Popularity	Popularity_R	Popularity_DR
Boy	Noah	120	1	1	1
Boy	Ethan	115	2	2	2
Girl	Emma	106	3	3	3
Boy	Jacob	101	4	4	4
Girl	Isabella	100	5	5	5
Girl	Olivia	100	6	5	5
Girl	Ava	95	7	7	6
Girl	Sophia	88	8	8	7
Boy	Liam	84	9	9	8
Boy	Logan	73	10	10	9

Bisa dilihat terdapat perbedaan antara kolom Popularity, Popularity_R, dan Popularity_DR.

- Popularity:
  Mengurutkan nilai total terbesar hingga terkecil tanpa memperhatikan kesamaan nilai total.
- Popularity_R atau RANK:
  Mengurutkan data berdasarkan nilai **Total** dari yang terbesar ke terkecil. Jika terdapat **nilai Total yang sama**, maka data tersebut akan memperoleh **peringkat (rank) yang sama**. Namun, **peringkat berikutnya akan melewati (skip) sejumlah peringkat yang telah digunakan**.

  Contoh pada gambar:

  - Isabella memiliki Total **100** → Rank **5**
  - Olivia memiliki Total **100** → Rank **5**
  - Karena ada dua data yang berada di peringkat 5, maka peringkat berikutnya **bukan 6**, melainkan **7** (Ava).

  Urutan RANK() menjadi:
  1, 2, 3, 4, 5, 5, 7, 8, 9, 10

- Popularity_DR atau DENSER_RANK:
  Mengurutkan data berdasarkan nilai **Total** dari yang terbesar ke terkecil. Jika terdapat **nilai Total yang sama**, maka data tersebut juga akan memperoleh **peringkat yang sama**. Namun, **peringkat berikutnya tidak melewati angka (tidak ada gap)** sehingga ranking tetap berurutan.

Contoh pada gambar:

- Isabella memiliki Total **100** → Dense Rank **5**
- Olivia memiliki Total **100** → Dense Rank **5**
- Data berikutnya, Ava (95), memperoleh **Dense Rank 6**, bukan 7.

  Urutan DENSE_RANK() menjadi:
  1, 2, 3, 4, 5, 5, 6, 7, 8, 9

Dengan CTE:

```
WITH ranked_names AS (
   SELECT Gender,
        Name,
        Total,
        ROW_NUMBER() OVER(
           ORDER BY Total DESC
        ) AS Popularity,
        RANK() OVER(
           ORDER BY Total DESC
        ) AS Popularity_R,
        DENSE_RANK() OVER(
           ORDER BY Total DESC
        ) AS Popularity_DR
   FROM baby_names
)

SELECT *
FROM ranked_names;
```

8. Mencoba menggunakan windows yang berbeda

Kita dapat menggunakan windows lain yaitu PARTITION by untuk mempartisi bagian gender perempuan dengan laki-laki dan juga ranking populasi diurutkan berdasarkan gender.

Contoh:

```
select Gender, Name, Total,
        row_number() over(partition by gender order by total desc) as Popularity
from baby_names;
```

```
42      -- 5. TRY DIFFERENT WINDOWS
43      select Gender, Name, Total,
44              row_number() over(partition by gender order by total desc) as Popularity
45      from baby_names;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Gender	Name	Total	Popularity
Boy	Noah	120	1
Boy	Ethan	115	2
Boy	Jacob	101	3
Boy	Liam	84	4
Boy	Logan	73	5
Girl	Emma	106	1
Girl	Isabella	100	2
Girl	Olivia	100	3
Girl	Ava	95	4
Girl	Sophia	88	5

9. TOP 3 Nama dengan nilai Total terbesar berdasarkan gender

```
select * from
(select Gender, Name, Total,
        row_number() over(partition by gender order by total desc) as Popularity
from baby_names) as popularity
where popularity <=3;
```

```
47      -- 6. WHAT ARE THE TOP 3 MOST POPULAR NAME FOR EACH GENDER?
48      select * from
49
50      (select Gender, Name, Total,
51              row_number() over(partition by gender order by total desc) as Popularity
52      from baby_names) as popularity
53      where popularity <=3;
54
```

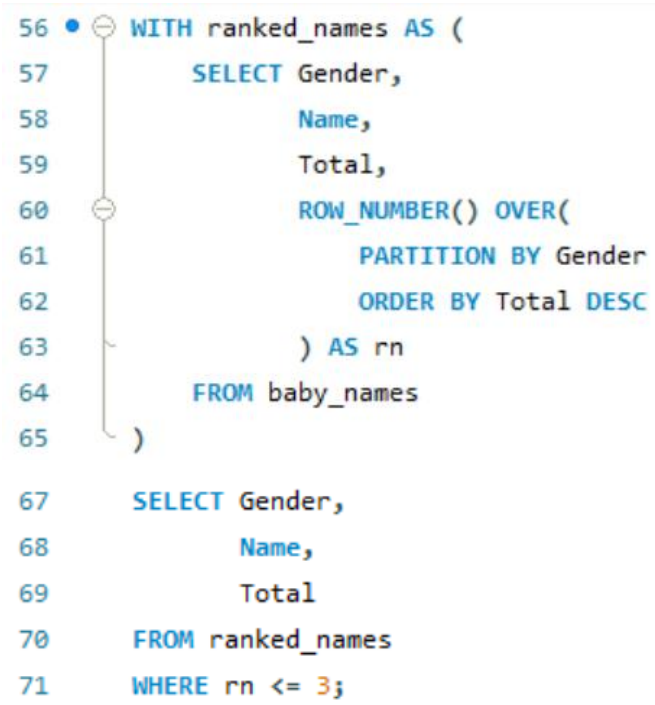
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

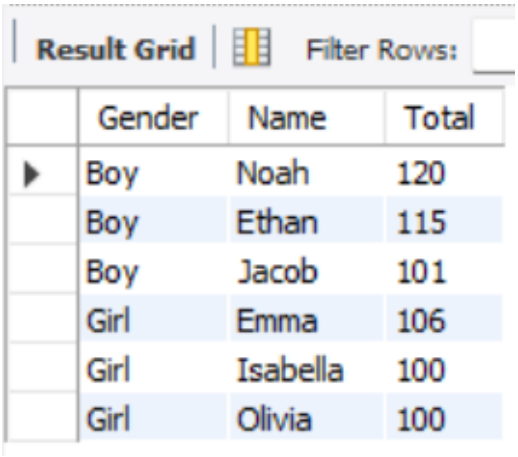
Gender	Name	Total	Popularity
Boy	Noah	120	1
Boy	Ethan	115	2
Boy	Jacob	101	3
Girl	Emma	106	1
Girl	Isabella	100	2
Girl	Olivia	100	3

Selain Nested subquery seperti di atas, kita bisa buat dengan CTE (Common Table Expression)

```
WITH ranked_names AS (
    SELECT Gender,
        Name,
        Total,
        ROW_NUMBER() OVER(
            PARTITION BY Gender
            ORDER BY Total DESC
        ) AS rn
    FROM baby_names
)

SELECT Gender,
    Name,
    Total
FROM ranked_names
WHERE rn <= 3;
```





	Gender	Name	Total
▶	Boy	Noah	120
	Boy	Ethan	115
	Boy	Jacob	101
	Girl	Emma	106
	Girl	Isabella	100
	Girl	Olivia	100